Enterprise BIM Platform

Nexus BIM Automation Hub — TypeScript Architecture

Complete Full-Stack TypeScript System Blueprint (Next.js 14 App Router, React, Tailwind CSS, Prisma, SQLite/PostgreSQL)

1. TYPESCRIPT DIRECTORY ARCHITECTURE

The enterprise BIM platform structured for full-stack TypeScript with Next.js 14, React, Tailwind CSS, Prisma, and Zod validation.

```
nexus-bim-hub-ts/
├── package.json
├── tsconfig.json
├── tailwind.config.ts
├── prisma/
│   └── schema.prisma
├── src/
│   ├── app/                    # Next.js App Router (Pages & API Routes)
│   │   ├── layout.tsx
│   │   ├── page.tsx            # Login Page
│   │   ├── dashboard/page.tsx
│   │   ├── buildings/page.tsx
│   │   ├── floors/page.tsx
│   │   ├── structural/page.tsx
│   │   ├── models/page.tsx
│   │   ├── reports/page.tsx
│   │   ├── ai-assistant/page.tsx
│   │   ├── settings/page.tsx
│   │   └── api/                # REST API Endpoints
│   │       ├── auth/route.ts
│   │       ├── buildings/route.ts
│   │       ├── models/route.ts
│   │       └── structural/route.ts
│   ├── components/             # React / Tailwind Components
│   │   ├── ui/                 # Reusable Primitives
│   │   ├── Navbar.tsx
│   │   ├── Sidebar.tsx
│   │   ├── KpiCard.tsx
│   │   ├── HealthChart.tsx
│   │   └── ModelUploader.tsx
│   ├── lib/                    # Core Infrastructure & Utilities
│   │   ├── db.ts               # Prisma ORM Client
│   │   ├── auth.ts             # Auth Strategy
│   │   └── utils.ts            # Helpers
│   └── types/                  # Strictly Typed Domain Interfaces
│       └── index.ts
└── public/
    └── assets/
```

2. STRICT DOMAIN TYPES DEFINITION (`SRC/TYPES/INDEX.TS`)

File: src/types/index.ts

```typescript
export type UserRole = 'Lead BIM Architect' | 'Facility Manager' | 'Inspector';

export interface User {
  id: number;
  username: string;
  fullName: string;
  email: string;
  role: UserRole;
}

export interface Building {
  id: number;
  code: string;
  name: string;
  campus: string;
  manager: string;
  status: 'Active' | 'Under Inspection' | 'Decommissioned';
}
```

```typescript
export interface StructuralElement {
  id: number;
  elementTag: string;
  buildingId: number;
  category: string;
  material: string;
  healthIndex: number;
  status: string;
  lastInspectionDate: string;
}

export interface BimModel {
  id: number;
  fileName: string;
  format: 'IFC' | 'RVT' | 'DWG' | 'NWD' | 'PDF';
  fileSizeMb: number;
  buildingId: number;
  uploadedBy: string;
  version: string;
  status: string;
}
```

3. PRISMA DATABASE SCHEMA (`PRISMA/SCHEMA.PRISMA`)

File: prisma/schema.prisma

```
datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id       Int    @id @default(autoincrement())
  username String @unique
  password String
  fullName String
  email    String
  role     String
}

model Building {
  id         Int                @id @default(autoincrement())
  code       String             @unique
  name       String
  campus     String
  manager    String
  status     String
  elements   StructuralElement[]
  models     BimModel[]
}

model StructuralElement {
  id                Int      @id @default(autoincrement())
  elementTag        String   @unique
  buildingId        Int
  building          Building @relation(fields: [buildingId], references: [id])
  category          String
  material          String
  healthIndex       Int
  status            String
  lastInspectionDate String
}

model BimModel {
  id         Int      @id @default(autoincrement())
  fileName   String
  format     String
  fileSizeMb Float
  buildingId Int
  building   Building @relation(fields: [buildingId], references: [id])
  uploadedBy String
  version    String
```

```
  status     String
  uploadDate DateTime @default(now())
}
```

4. DATABASE CLIENT (`SRC/LIB/DB.TS`)

File: src/lib/db.ts

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = global as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma ||
  new PrismaClient({
    log: ['query'],
  });

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

5. REACT DASHBOARD TELEMETRY COMPONENT (`SRC/COMPONENTS/HEALTHCHART.TSX`)

File: src/components/HealthChart.tsx

```
import React from 'react';
import { StructuralElement } from '@/types';

interface Props {
  elements: StructuralElement[];
}

export const HealthChart: React.FC<Props> = ({ elements }) => {
  return (
    <div className="bg-slate-800/80 border border-slate-700 rounded-xl p-5 shadow-lg">
      <h3 className="text-lg font-bold text-sky-400 mb-4">Structural Health Telemetry</h3>
      <div className="space-y-3">
        {elements.map((el) => {
          const isHealthy = el.healthIndex >= 90;
          const barColor = isHealthy ? 'bg-emerald-500' : el.healthIndex >= 75 ? 'bg-amber-500' : 'bg-red-500';

          return (
            <div key={el.id} className="space-y-1">
              <div className="flex justify-between text-xs text-slate-300">
                <span className="font-mono font-bold text-sky-300">{el.elementTag}</span>
                <span className="font-bold">{el.healthIndex}%</span>
              </div>
              <div className="w-full bg-slate-900 rounded-full h-2.5 overflow-hidden">
                <div
                  className={`h-2.5 rounded-full ${barColor}`}
                  style={{ width: `${el.healthIndex}%` }}
                />
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
};
```

6. API ROUTE HANDLER (`SRC/APP/API/BUILDINGS/ROUTE.TS`)

File: src/app/api/buildings/route.ts

```
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/db';

export async function GET() {
  try {
    const buildings = await prisma.building.findMany({
      include: { models: true, elements: true },
    });
    return NextResponse.json(buildings);
```

```
  } catch (error) {
    return NextResponse.json({ error: 'Failed to fetch facilities' }, { status: 500 });
  }
}

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const newBuilding = await prisma.building.create({
      data: {
        code: body.code,
        name: body.name,
        campus: body.campus,
        manager: body.manager,
        status: 'Active',
      },
    });
    return NextResponse.json(newBuilding, { status: 201 });
  } catch (error) {
    return NextResponse.json({ error: 'Failed to create facility' }, { status: 400 });
  }
}
```

7. TECH STACK MAPPING (PYTHON/STREAMLIT VS TYPESCRIPT)

Layer	Previous Python Setup	TypeScript Replacement Stack
Frontend UI	Streamlit (`st.markdown`, `st.columns`)	React 18 + Next.js 14 + Tailwind CSS
Backend Engine	Python / Streamlit Runner	Next.js API Routes (Node.js/Edge runtime)
ORM / Database	`sqlite3` / Raw SQL Queries	Prisma ORM / Drizzle ORM (Type-Safe)
Visualizations	Plotly Express (`plotly.express`)	Recharts / Chart.js / `@observablehq/plot`
Data Validation	Manual Python checks	Zod (`import { z } from 'zod'`)